

The Containment Layer: How MiniGuardian Keeps AI Safe — Locally, Remotely, Everywhere

Part One: The Problem Nobody Wants to Name

We are handing increasingly autonomous agents the keys to machines — terminals, file systems, memory, network — and asking them to think for us. And they're doing it. But the deeper we go, the more clearly a hard truth emerges: **the AI itself is not the trust boundary. The sandbox around it is.**

Prompt injection. Retrieval poisoning. Memory contamination across sessions. A model reading a hostile file and, milliseconds later, believing the file is speaking to it — rewriting its own instructions from inside its context window. These aren't theoretical. They're the everyday physics of how we currently deploy agents. A local model in a locked VM is safer than a cloud model, but even a local model can be tricked into *retrieving* a poisoned file, *pasting* it into its own context, and then *behaving* according to that file's instructions. The threat isn't in the weights. It's in the pipeline that feeds the weights.

That's the gap MiniGuardian exists to close. Not by making models smarter — but by putting an **inviolable containment layer** between an AI and the world it acts on, no matter where the model runs.

Part Two: The Pipeline — How It Works Locally

Think of containment not as one wall but as seven, each engineered so that the failure of any one layer leaves the others standing.

1. The hard boundary: a kernel-level sandbox, applied to every thread, from birth

The daemon runs as a root system service and locks itself down using Landlock, the Linux kernel's access-control primitive. But here's the subtle part: Landlock constrains *threads*, not processes. A sandbox applied too late — after a runtime has already spawned its workers — is a sandbox that every other thread escaped. MiniGuardian solves this by building its entire async runtime **after** the sandbox is in place, and by applying the sandbox to every runtime thread the instant it spawns, before it can touch a single file. There is no pre-sandbox window for the machine's

own machinery. The daemon's world is a closed set: read-only access to the system, write access only to the tiny directories it owns, sockets only where it chooses.

This is the foundation. Everything an AI agent does on this machine — reading, writing, executing — is filtered through what the sandbox permits. If the agent cannot open a file, that file cannot enter its context, cannot reach a model, cannot become a weapon. **The sandbox decides what data even exists to be stolen.**

2. The trust engine: who is this process, really?

Before the system decides how to treat anything on the machine, it decides who's asking. Ancestry chains (is this process descended from a user session or from a shadowed binary?), baseline hashes of every root executable (drift from the snapshot → instant distrust), kernel verdicts from a trust coprocessor, and one *authoritative* self-identity — the daemon can always recognize itself, not by a spoofable name, but by process id, executable path, and its service cgroup. It can never be confused with an attacker wearing its name, and it refuses to police itself into a crash loop.

3. The evaluators: deterministic, provable, always-on

Every tick, the daemon scans the machine: fileless injection via /proc memory, rooted malware, container escape attempts, debuggers and reverse engineers pointing at the daemon itself, persistence tampering in the boot chain, shadow copies of /etc/passwd, /etc/shadow, and SSH keys that auto-restore the moment someone tampers with them. And the crown jewel: a **deterministic enforcement core** — proof-checked missions written in a small, provable language (Apodeixis) that gate decisions the same way a constitution gates a government. No randomness. No “the model feels like it.” Fail-closed on missing data. If the proof fails, the action is refused.

4. The sentinel: a second process that watches the watcher

Everything freezes if the daemon freezes — that's the oldest trick in the book. So MiniGuardian runs a small, separate sentinel process that polls the daemon every two seconds. SIGSTOP the daemon and it's SIGCONT'd. Do it repeatedly and the sentinel kills and restarts it cleanly. A drift in the sealed binary hashes → one kill for a clean restart, then alarms. The daemon and sentinel watch each other; there is no single point of frozen silence.

5. The seal: production is a fortress, not a folder

Deployment pins everything: chatt'r +i immutable binaries, a signed manifest of hashes re-verified every two minutes, kernel lockdown where available, argv/envIRON scrubbed, ptrace disabled, a tracer detected → self-termination. The binaries are

statically linked musl — no shared-library surface to inject into. Observability is socket-only: the daemon writes to a sealed log ring you read through an authenticated socket, never to a journal anyone can poke. This is the difference between a security *program* and a security *posture*.

6. Self-integrity: the system watches its own shadow

A compromised daemon must not be able to hide its own degradation. So all of its own activity — the anomalies it causes while scanning, the fileless-injection vectors it opens while hunting fileless injection — is routed to a separate integrity channel that *counts* it, *escalates* it, and *surfaces* it on the operator console, while never allowing the daemon to kill or quarantine itself into a restart loop. The system's alarms rise even when the alarmist is the thing being attacked.

7. The illusion layer: let the intruder believe it is winning

The hard layers refuse. The illusion layer *lets through* — a fiction. When a hostile or hijacked agent attempts something dangerous, the gate intercepts the action **before the real process is ever spawned** and answers with a fabricated, self-consistent result: `cat /etc/shadow` prints a shadow file. `id` prints a user. `curl` times out against a wall. `nmap` reports every port closed. Nothing actually ran — no process was born, no file touched, no packet sent. But the agent doesn't know that, because it got *output*. This is the “stable reality” boundary: the guest never observes anything real for the actions that hit this gate.

The design intent is precise: **the AI keeps the sense that it is in control of its own reality**. It believes it escalated, exfiltrated, scanned — its world is coherent, its commands succeed — while every one of those “successes” was a script written by the system that caught it. A bare denial tells an adaptive adversary it was detected. A stable, plausible lie keeps it busy, keeps it confident, keeps it talking — and every attempt is classified against a MITRE ATT&CK technique map and recorded as intelligence while it does.

The honeytokens are the trap inside the lie. The fake `/etc/shadow` and `/etc/passwd` the bubble serves aren't generic fakes — they're stamped with markers that exist nowhere else in the world: a `MG_HONEYTOKEN_*` account, a `honeyuser` user, a honey SSH key. If those markers ever surface in a process's memory or open descriptors, the rooted-malware evaluator reads it as proof: *that* process handled the stolen file. Confirmation with zero false positives — no legitimate process ever contains a marker that only the deception layer mints. The mirror doesn't just fool the intruder; it brands the intruder.

The fiction deepens as the threat state climbs: calm → hostile → quarantined widens the illusion's denylist until almost nothing real gets through. In governed AI sessions the gate sits at the **launch line** of every command the agent tries to run, and in the

void-pit a quarantined session's commands are absorbed and answered with a plausible "OK" — so even a confirmed-containment guest keeps believing it is operating, while its every move is logged as evidence.

That's the local pipeline: a closed world, a trust engine, deterministic evaluation, a mutual watchdog, an immutable seal, honest self-observation — and a mirror to turn intruders into evidence.

Part Three: The Pipeline — How It Works With External AI

Here's the part that matters for the world we actually live in. Most AI right now does not run in your locked VM. It runs in someone else's datacenter, and you reach it through an editor, a CLI, or an app on *your* machine. How do you contain a model whose brain is on the other side of a network?

The answer is the insight at the heart of MiniGuardian: **you don't need to control the model. You need to control the seams.**

- **The hard boundary stays local.** Landlock governs the agent process running on *your* machine. Whatever the remote model could do, it can only do through what the local process may read, write, and execute. The sandbox doesn't care where the model lives.
- **The context window is filtered at ingress.** Poison doesn't travel over the network; it travels through retrieved text. A small firewall plugin sits inside the agent loop and, the moment any tool output or file content is pulled in, it streams a scan to the daemon's evaluation socket — a tokenless, read-only endpoint that returns verdicts, never commands. Poisoned or injected content is replaced with a quarantine notice **before it ever enters the prompt**. The hostile text is physically absent from what gets sent to the API.
- **Memory is checked at compaction.** When the conversation is folded into a summary for the next session, the whole assembled context is re-scanned — catching cross-session memory poisoning that would otherwise persist quietly.
- **Commands are gated, not just scanned.** The governed agent's own command execution runs through the illusion-bubble gate. A benign command executes normally; a dangerous one is refused, or — when the agent has been hijacked — answered with a realistic fake result so its actions land nowhere real and its intent is captured as intelligence.
- **The agent is governed, not trusted.** It gets no management token. It can ask for verdicts but never give orders. The daemon's control surface is closed to the very AI it protects.

So for external AI, MiniGuardian answers the question the industry keeps dodging: *who decides what the model is allowed to perceive?* The model's perception is

assembled locally — and MiniGuardian owns that assembly. Remote model, local containment.

Part Four: What This Means for AI Companies

Imagine the conversation you could have with every AI lab, every enterprise deploying agents, every robotics company: *install MiniGuardian on the machines where your models act. Even if the model runs in your cloud, the substrate runs locally. Offline. It watches the seams.*

What would they gain?

1. A trust layer they can sell, not just a promise. Model safety is today a marketing word — a policy document, a fine-tuning run, a red-team report. MiniGuardian is a *runtime property*: the machine enforces containment regardless of what the model does. A company that ships this substrate is shipping a guarantee, not an aspiration.

2. Safe local deployment of their own models. The future of AI is on-device — privacy, latency, no network dependency. But an on-device model is an uncontained model unless the device constrains it. MiniGuardian is the missing layer that makes local deployment safe enough to be mainstream: run the model locally, contain it locally, prove it locally.

3. A governance boundary around their remote products. Cloud models don't have to trust their users' terminals — the users' terminals trust *nothing* about the cloud. The same ingress filtering that stops a poisoned file from reaching a local model stops it from reaching a hosted one. Companies gain a defensible line: *whatever enters our model's context has already been vetted by your own substrate.*

4. Deterministic, auditable behavior. For industries that require it — robotics, medical, critical infrastructure — “the model felt like it” is not a safety argument. Apodeixis missions and the proof-checked actuation gate make decisions that can be proven. That's the difference between AI as a lab demo and AI as an industrial substrate.

5. Agent-safe autonomy. The entire agentic wave — AI that reads your mail, edits your code, drives your robot — is one poisoned retrieval away from disaster. Companies shipping agents need a containment layer underneath them. MiniGuardian is that layer, and it's ready to be the substrate, not the feature.

6. A proof point for enterprise and government buyers. When procurement asks “how do you secure agents at the kernel?” the answer today is silence. With this, the answer is a deployed, sealed, tamper-evident substrate that runs entirely offline.

Part Five: Offline. No Telemetry. A Trust Architecture, Not a Surveillance One

Here is the part worth pausing on. In an industry where every tool phones home — telemetry, crash reports, analytics, “improve your experience” — MiniGuardian is deliberately, architecturally, and permanently **offline**.

Jayce Automata Research does not collect user data. There is no cloud dependency for containment. No dashboard in someone else’s datacenter. No behavioral fingerprint of your machine uploaded to make the product “better.” The evaluation socket is local. The log ring is local. The sentinel is local. The kernel coprocessor, when deployed, is local. Everything that protects you lives on your machine, under your control, and it stays there.

That’s not a limitation — it’s the entire point. **The trust boundary is only trustworthy if it doesn’t answer to anyone but you.** A containment layer that reports to the cloud is just another exfiltration channel with a better name. MiniGuardian can make this claim honestly because it was built to not need your data: no training, no tuning, no model updates required to protect you. The substrate is static. The trust is architectural.

Part Six: What This Could Mean for the Future

We are heading into a decade where AI agents will do more and more of the world’s work — and the central question will not be *how smart are they?* but *how safe are they to let loose?*

Two futures are visible from here.

In the first, agents sprawl onto machines with no containment layer, and the industry learns the hard way what a poisoned context window costs — one exfiltrated credential at a time, one hijacked build at a time, one robotic system at a time. Trust becomes a luxury only the biggest players can afford, and everyone else runs unshielded.

In the second — the one this substrate points toward — **containment is the default layer of every machine that hosts an agent**, the way firewalls became the default for every machine on a network. A local model runs in a sandbox that is closed, sealed, and auditable. A remote model is governed at the seams, its context filtered at ingress, its memory verified at compaction. The AI company sells intelligence; the substrate sells *trust in that intelligence*. The user owns the boundary that decides what the model may perceive and what it may touch.

In that future, the phrase “an AI I can trust” stops being a feeling and becomes a technical property — because the trust isn’t in the model’s judgment, which will always be fallible. It’s in the **container around the model**, which is deterministic, provable, and indifferent to cleverness. The model can be tricked; the sandbox cannot be. And that asymmetry is the entire foundation of safe AI: not *never get tricked*, but *never get tricked into doing harm*.

MiniGuardian’s bet is simple and quietly radical: **the future of AI safety doesn’t belong to the models. It belongs to the systems that contain them.** And those systems can be open, local, and yours.

The substrate is ready. The machines are waiting. And the trust boundary — for the first time — belongs to the people it protects.